

Instruções virtuais para processamento paralelo



UNIFEI

Universidade Federal de Itajubá

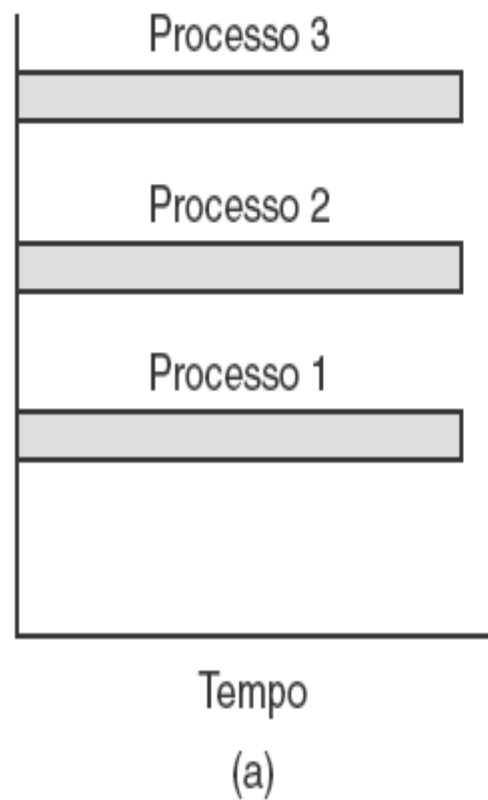
IMC – Instituto de Matemática e Computação

Prof. Carlos Minoru Tamaki

Instruções virtuais para processamento paralelo

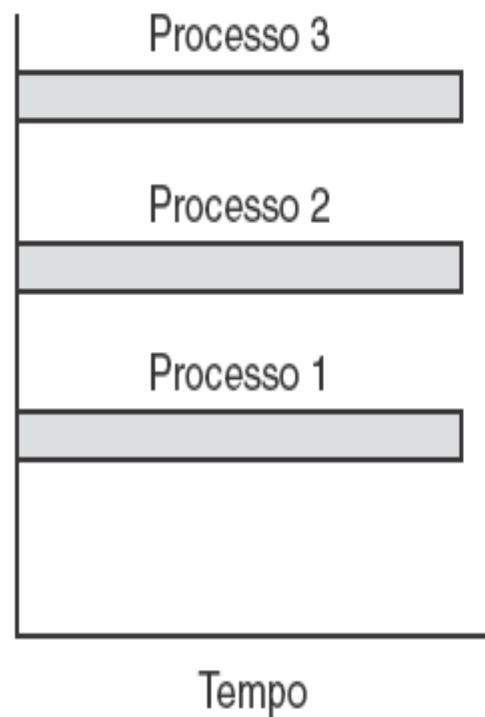
- Algoritmos que podem ser divididos para execução em dois ou mais processadores
- Algoritmos que divididos em porções podem executar em paralelo e diminuir o tempo de execução
- Einstein => é impossível transmitir sinais elétricos a cima da velocidade da luz, 30 cm/ns no vácuo
- Se a CPU precisar de dados da memória distante mais de 30 cm vai levar mais que 1 ns

(a) Processamento
verdadeiramente
paralelo com várias
CPUs.

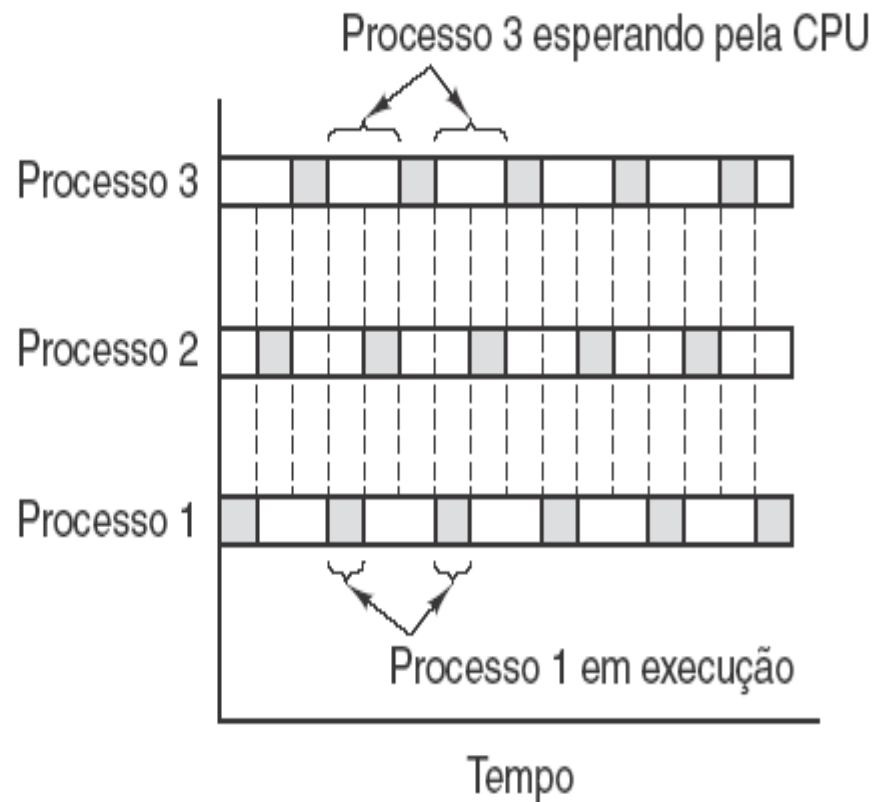


(a) Processamento verdadeiramente paralelo com várias CPUs.

(b) Processamento paralelo simulado pela comutação dinâmica de uma única CPU entre três processos.



(a)



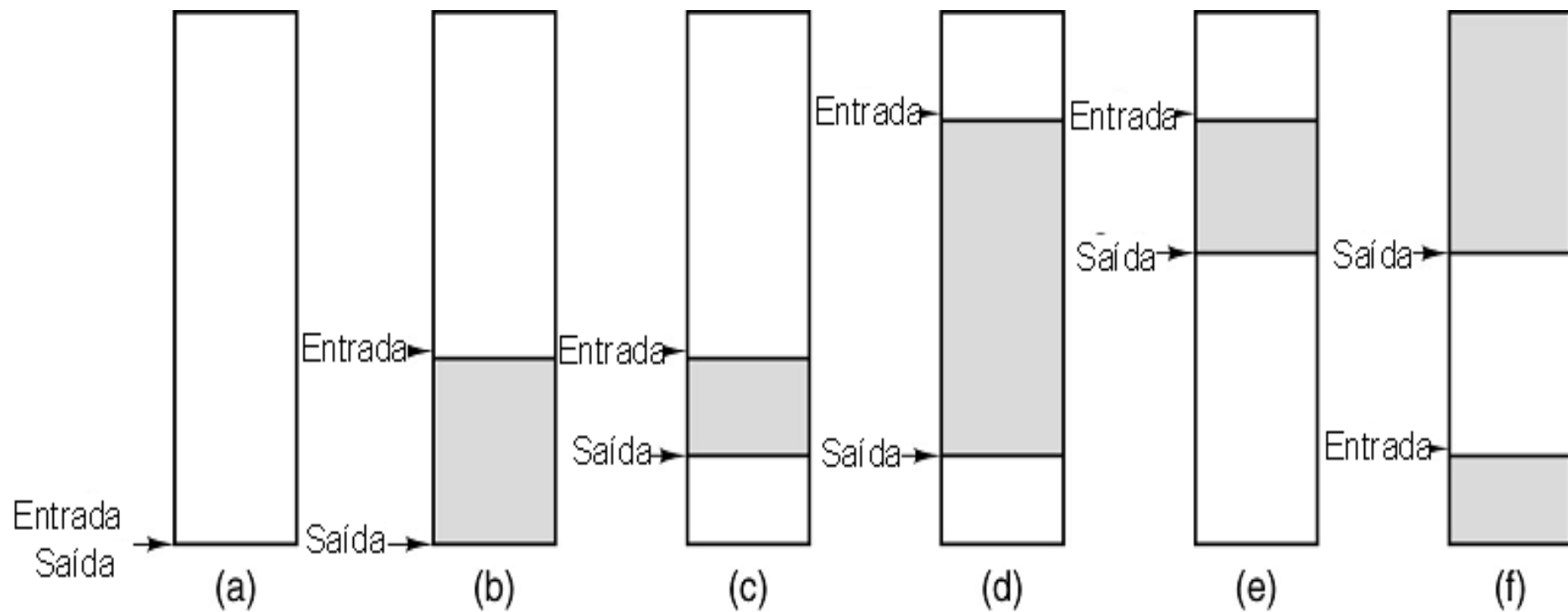
(b)

Criação de processo

- Um programa é executado sempre como parte de um processo
- Um processo é caracterizado por um estado e um espaço de endereçamento
- O estado inclui pelo menos o contador de programa, um ponteiro de pilha e os registradores gerais
- Os sistemas operacionais modernos permitem que processos sejam criados e encerrados dinamicamente

Condição de disputa

- Processos paralelos precisam se comunicar e sincronizar
- A sincronização de processos e suas dificuldades serão explicadas
- Considere a seguinte situação: existem 2 processos independentes, o processo 1, **produtor** e o processo 2, **consumidor**, que se comunicam por meio de um buffer compartilhado na memória. O produtor calcula números primos e coloca no buffer um por vez. O consumidor retira esses números do buffer um por vez e os imprime



Uso de um buffer circular

Processamento
paralelo com uma
condição fatal de
disputa.

Principal em Java

```
public class m {  
    final public static int BUF_SIZE = 100;           // buffer executa de 0 a 99  
    final public static long MAX_PRIME = 1000000000000L; // pare aqui  
    public static int in = 0, out = 0;                // ponteiros para os dados  
    public static long buffer[ ] = new long[BUF_SIZE]; // primos armazenados aqui  
    public static producer p;                         // nome do produtor  
    public static consumer c;                         // nome do consumidor  
  
    public static void main(String args[ ]) {          // classe principal  
        p = new producer( );                          // crie o produtor  
        c = new consumer( );                          // crie o consumidor  
        p.start( );                                    // inicie o produtor  
        c.start( );                                    // inicie o consumidor  
    }  
  
    // Essa é uma função para incrementar in e out de modo cíclico  
    public static int next(int k) {if (k < BUF_SIZE - 1) return(k+1); else return(0);}  
}
```


Processamento
paralelo com uma
condição fatal de
disputa.

Produtor em Java

```
class producer extends Thread {  
    public void run( ) {  
        long prime = 2;  
  
        while (prime < m.MAX_PRIME) {  
            prime = next_prime(prime);  
            if (m.next(m.in) == m.out) suspend( );  
            m.buffer[m.in] = prime;  
            m.in = m.next(m.in);  
            if (m.next(m.out) == m.in) m.c.resume( );  
        }  
    }  
  
    private long next_prime(long prime){ ... }  
}
```

// classe produtor
// código do produtor
// variável transitória

// declaração P1
// declaração P2
// declaração P3
// declaração P4
// declaração P5

// função que calcula o próximo primo

Processamento
paralelo com uma
condição fatal de
disputa.

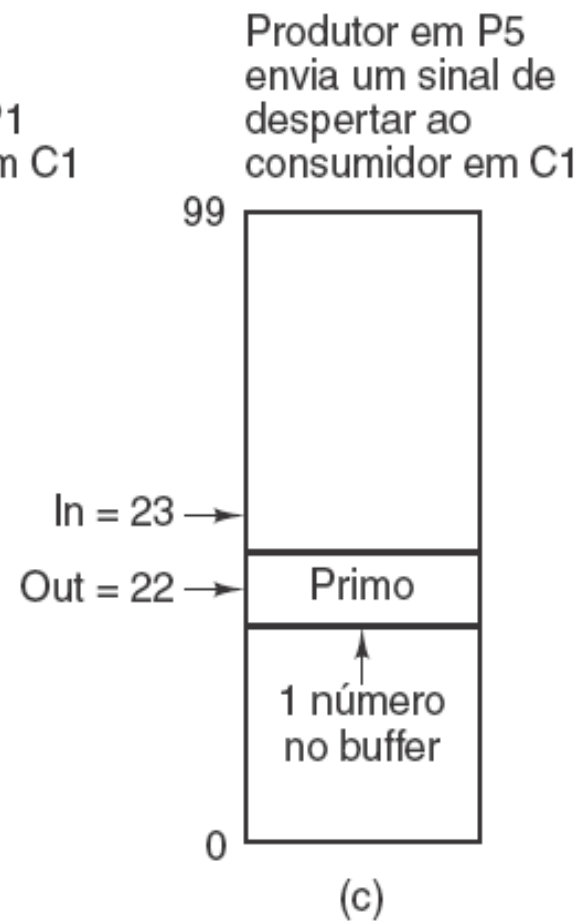
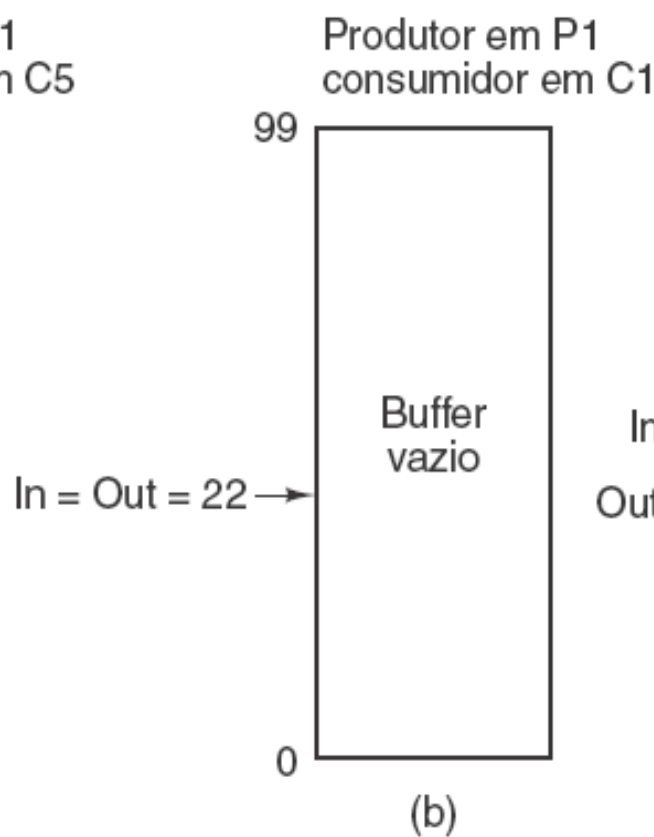
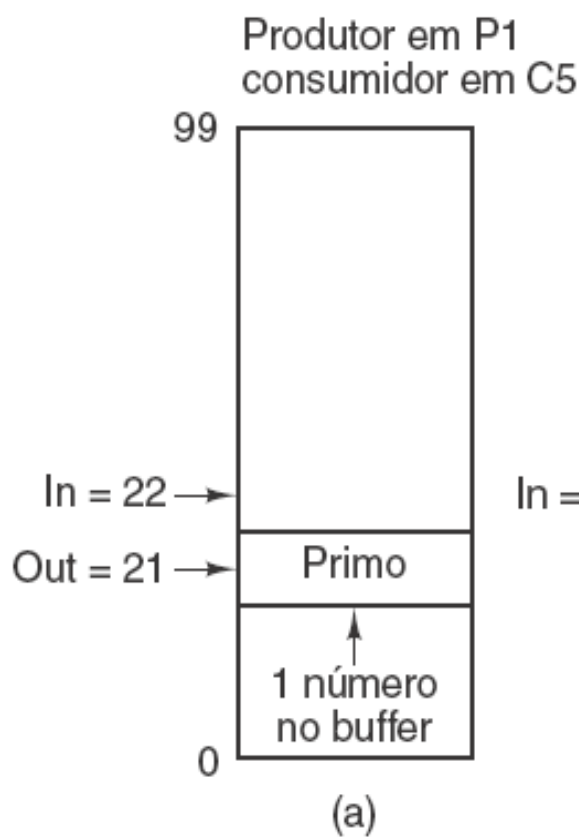
Consumidor em Java

```
class consumer extends Thread {                                // classe consumidor
    public void run( ) {                                         // código do consumidor
        long emirp = 2;                                         // variável transitória

        while (emirp < m.MAX_PRIME) {
            if (m.in == m.out) suspend( );                      // declaração C1
            emirp = m.buffer[m.out];                             // declaração C2
            m.out = m.next(m.out);                               // declaração C3
            if (m.out == m.next(m.next(m.in))) m.p.resume( );   // declaração C4
            System.out.println(emirp);                           // declaração C5
        }
    }
}
```

Tanenbaum, A.S., Austin, T.; Organização Estruturada de Computadores; São Paulo; Pearson; 6ª Ed, 2013

Falha do mecanismo de comunicação produtor-consumidor.



Sincronização de processos usando semáforos

Efeitos de uma operação de semáforo

Instrução	Semáforo = 0	Semáforo > 0
Up	Semáforo = semáforo + 1; se o outro processo foi interrompido na tentativa de concluir uma instrução down nesse semáforo, agora ele pode concluir down e continuar executando	Semáforo = semáforo + 1
Down	Processo pára até que o outro processo execute uma instrução up nesse semáforo	Semáforo = semáforo - 1

Tanenbaum, A.S., Austin, T.; Organização Estruturada de Computadores; São Paulo; Pearson; 6^a Ed, 2013

Principal em Java usando semáforos

```
public class m {  
    final public static int BUF_SIZE = 100;           // buffer vai de 0 a 99  
    final public static long MAX_PRIME = 1000000000000L; // pare aqui  
    public static int in = 0, out = 0;                // ponteiros para os dados  
    public static long buffer[ ] = new long[BUF_SIZE]; // primos armazenados aqui  
    public static producer p;                         // nome do produtor  
    public static consumer c;                         // nome do consumidor  
    public static int filled = 0, available = 100;    // semáforos  
  
    public static void main(String args[ ]) {         // classe principal  
        p = new producer( );                        // crie o produtor  
        c = new consumer( );                        // crie o consumidor  
        p.start( );                                  // inicie o produtor  
        c.start( );                                  // inicie o consumidor  
    }  
    // Essa é uma função para incrementar in e out de modo cíclico  
    public static int next(int k) {if (k < BUF_SIZE - 1) return(k+1); else return(0);}  
}
```

Produtor em Java usando semáforos

```
class producer extends Thread {  
    native void up(int s); native void down(int s);  
    public void run( ) {  
        long prime = 2;  
  
        while (prime < m.MAX_PRIME) {  
            prime = next_prime(prime);  
            down(m.available);  
            m.buffer[m.in] = prime;  
            m.in = m.next(m.in);  
            up(m.filled);  
        }  
    }  
  
    private long next_prime(long prime){ ... }  
}
```

// classe produtor
// métodos em semáforos
// código do produtor
// variável transitória

// declaração P1
// declaração P2
// declaração P3
// declaração P4
// declaração P5

// função que calcula o próximo primo

Consumidor em Java usando semáforos

```
class consumer extends Thread {  
    native void up(int s); native void down(int s);  
    public void run( ) {  
        long emirp = 2;  
  
        while (emirp < m.MAX_PRIME) {  
            down(m.filled);  
            emirp = m.buffer[m.out];  
            m.out = m.next(m.out);  
            up(m.available);  
            System.out.println(emirp);  
        }  
    }  
}
```

// classe consumidor
// métodos em semáforos
// código do consumidor
// variável transitória

// declaração C1
// declaração C2
// declaração C3
// declaração C4
// declaração C5

Bibliografia

- Organização Estruturada de Computadores – Andrew S. Tanenbaum, Tood Austin – 6ª Edição 2013 Prentice Hall
- Vários sites da internet